

同濟大學

TONGJI UNIVERSITY

毕业设计（论文）

课题名称 基于卷积神经网络的 MNIST 手写数字训练

副标题 人工智能导论课程作业

学院 计算机科学与技术学院

专业 软件工程

学生姓名 郭炫君

学号 2452654

指导教师 邓浩

日期 2026 年 3 月 18 日

装

订

线

基于卷积神经网络的 MNIST 手写数字识别

1 主要代码说明与算法流程

1.1 数据集准备

MNIST 数据集包含 60,000 张 28×28 像素的灰度手写数字图像作为训练集，10,000 张作为测试集。数据预处理包括两个步骤：

- (1) `ToTensor()`：将 PIL 图像转换为范围在 $[0,1]$ 的浮点型张量。
- (2) `Normalize((0.1307,), (0.3081,))`：使用 MNIST 数据集的全局均值和标准差进行标准化，使数据分布接近标准正态分布，有助于加速模型收敛。

`DataLoader` 设置批大小为 64，训练集和测试集随机打乱，便于批处理训练与评估。

1.2 神经网络架构

(1) 卷积层是 CNN 的核心组件之一。它通过在图像上滑动卷积核来提取图像的特征。每个卷积核可以学习不同的特征模式。本实验中，我们定义了两个卷积层。

(2) 池化层用于减少卷积层输出的空间维度，同时保留重要信息。在本实验中，我们采用最大池化 (`MaxPool2d`) 操作，将每个 2×2 的区域中的最大值作为输出。

(3) 全连接层用于将卷积层输出的特征图转换为分类结果。在本实验中，我们定义两个全连接层。

(4) 本实验中，我们使用激活函数 ReLU，在负数时直接归零，正数不变。我们使用了 3 次激活函数，分别在两个卷积层和第一个全连接层后面。

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        relu = nn.ReLU()
        self.module = nn.Sequential(
            Conv2d(in_channels=1, out_channels=32, kernel_size=5, padding=2, stride=1),
            relu,
            nn.MaxPool2d(2), # 28 -> 14
            Conv2d(in_channels=32, out_channels=32, kernel_size=5, padding=2, stride=1),
            relu,
            nn.MaxPool2d(2), # 14 -> 7
            Conv2d(in_channels=32, out_channels=64, kernel_size=5, padding=2, stride=1),
            relu,
            nn.MaxPool2d(2), # 7 -> 3
            Flatten(),
            nn.Linear(64 * 3 * 3, out_features=64),
            relu,
            nn.Linear(in_features=64, out_features=10), # 10 类 (0-9)
        )
```

图 1.1 卷积神经网络模型定义代码段

1.3 训练与测试流程

实验设定训练轮数（epoch）为 10，优化器选用随机梯度下降（SGD），学习率 0.01。损失函数为交叉熵损失（CrossEntropyLoss），适用于多分类任务。

1.3.1 训练主要步骤

- （1）将模型设为训练模式（`net.train()`）。
- （2）遍历训练数据加载器，获取图像和标签，并移至指定设备（GPU 或 CPU）。
- （3）前向传播计算输出，得到损失值。
- （4）优化器梯度清零，反向传播计算梯度，优化器更新参数。
- （5）累计每个样本的损失，用于计算平均训练损失。
- （6）每 100 步打印一次当前损失，并用 TensorBoard 记录。

1.3.2 测试主要步骤

测试阶段每轮结束后进行：

- （1）模型设为评估模式（`net.eval()`），关闭梯度计算。
- （2）遍历测试数据，前向传播计算损失，并累计总损失。
- （3）计算预测类别（`outputs.argmax(1)`），统计正确预测数量，得到准确率。
- （4）记录平均测试损失和准确率到 TensorBoard。

所有记录使用 SummaryWriter 保存，便于后续可视化分析。

```
# 8. 训练步骤
for i in range(epoch):
    print("第 {} 轮训练开始".format(i + 1))

    # 训练阶段
    net.train()
    total_train_loss = 0
    for data in train_loader:
        imgs, targets = data
        imgs = imgs.to(device)
        targets = targets.to(device)

        outputs = net(imgs)
        loss = loss_fn(outputs, targets)

        # 反向传播
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # 按样本累计 loss，后面再求平均
        total_train_loss += loss.item() * imgs.size(0)

    total_train_step += 1
```

图 1.2 训练步骤代码段

```
# 测试阶段
net.eval()
total_test_loss = 0
total_accuracy = 0
with torch.no_grad():
    for data in test_loader:
        imgs, targets = data
        imgs = imgs.to(device)
        targets = targets.to(device)

        outputs = net(imgs)
        loss = loss_fn(outputs, targets)
        # 按样本累计 loss，后面再求平均
        total_test_loss += loss.item() * imgs.size(0)

        # 计算预测正确的数量
        accuracy = (outputs.argmax(1) == targets).sum()
        total_accuracy += accuracy.item()
```

图 1.3 测试步骤代码段

2 实验结果分析

2.1 实验结果

2.1.1 训练损失变化

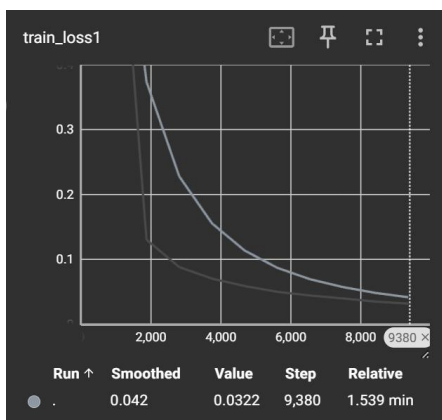


图 2.1 训练损失图像

从图 2.1 可以看出，训练损失整体呈下降趋势，初始阶段下降较快，后期趋于平缓。最终训练损失稳定在 0.0322 左右，表明模型在训练集上拟合良好，未出现明显欠拟合。

2.1.2 测试损失与准确率

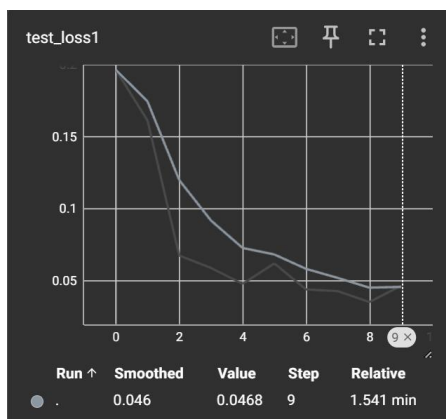


图 2.2 测试损失图像

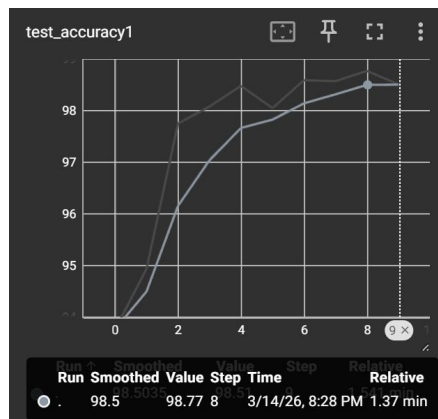


图 2.3 测试准确率图像

测试损失（图 2.2）从初始约 0.2 逐渐下降至第 9 步（epoch）时的 0.0468，说明模型泛化能力不断增强。测试准确率（图 2.3）同步提升，最终达到 98.50%，表明模型对未见过的数字图像具有较高的识别精度。训练过程中未观察到过拟合现象（测试损失未出现上升），模型参数设置

合理。

2.2 结果汇总

- （1）训练损失最终值：0.0322
- （2）测试损失最终值：0.0468
- （3）测试准确率最终值：98.50%

准确率接近 MNIST 任务常见水平（简单 CNN 通常可达 99%左右），本实验因网络深度适中、训练轮次有限，仍有微小提升空间。

3 总结

本次实验成功复现了基于 CNN 的手写数字识别 MNIST 任务，完整实现了数据加载、模型构建、训练与测试流程，并使用 TensorBoard 进行可视化监控。实验结果表明，所设计的卷积网络能够有效学习数字特征，在测试集上取得 98.50%的准确率，验证了 CNN 在图像分类任务中的强大能力。

通过本实验，我加深了对深度学习框架 PyTorch 的理解，掌握了模型训练的基本步骤及调参方法。后续可尝试调整网络结构（如增加 Batch Normalization、Dropout）、优化器（如 Adam）或数据增强，进一步提升模型性能。

参考文献

- [1] （美）阿斯顿·张（ASTON ZHANG），扎卡里·C.立顿（ZACHARY C.LIPTON），李沐（MULI），等. 动手学深度学习 PyTorch 版[M]. 北京：人民邮电出版社,2023..
- [2] 什么都不太会的研究生. 基于深度学习的 MNIST 手写数字数据集识别（准确率 99%，附代码）[EB/OL]. (2025-06-11)[2026-3-18]. https://blog.csdn.net/ps_hello/article/details/134169021.